Software Requirements Specification

for

HabitatHero

Version 1.0 approved

Prepared by TCE2, Group 2:

NG CAI XIAN

CHONG XIN LE

CHOW ZEN TENG

SIM YONG EN, IVAN

YEONG JIA WEN, JONATHAN

MUHAMMAD IZZUDDIN BIN HUSEIN

Nanyang Technological University

23/2/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 7
3. External Interface Requirements 7
3.1 User Interfaces 7
3.1.1 User Web Application Interface 7
3.1.2 Administrator Web Portal Interface 8
3.2 Hardware Interfaces 8
3.3 Software Interfaces 9
3.3.1 External APIs and Services 9
3.3.2 Internal Data Storage Interface 9
3.3.3 Runtime Environment Interfaces 9
3.4 Communications Interfaces 10
3.4.1 Network Protocols 10
3.4.2 Message Formatting 10
3.4.3 Communication Security 10
3.4.4 Synchronization Mechanisms 10
4. System Features 11
4.1 Hybrid User Profiling 11
4.2 Recommendation Engine Logic 12
4.3 Geospatial Analysis Features 13
4.4 Recommendation Engine Logic 15
4.5 System Administration 16
5. Other Nonfunctional Requirements 18
5.1 Performance Requirements 18
5.2 Safety Requirements 18
5.3 Security Requirements 19
5.4 Software Quality Attributes 19
5.5 Business Rules 19
6. Other Requirements 21
6.1 Database Requirements 21
6.2 Data Retention and Archival Requirements 21
6.3 Logging and Monitoring Requirements 21
6.4 Legal and Compliance Requirements 22
6.5 Internationalisation and Localisation Requirements 22
6.6 Deployment and Environment Requirements 22
7. Appendix A: Data Dictionary 23
8. Appendix B: Analysis Models 26

9. Appendix C: Use Case Description 27
10. Appendix D: To Be Determined List.....35

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Habitat Hero System, a decision-support web application that recommends HDB housing options based on users' housing constraints, livability preferences, and commute needs. The document serves as a contract between stakeholders and the development team, and provides a basis for design, implementation, and testing.

1.2 Document Conventions

Requirements are labelled and grouped as:

- REQ-x: Functional Requirements
- EIR-x: External Interface Requirements
- NFR-x: Non-Functional Requirements
- OR-x: Other Requirements

Keyword meanings:

- "shall" = mandatory requirement
- "should" = recommended but not strictly required
- "may" = optional behavior

1.3 Intended Audience and Reading Suggestions

This document is intended for multiple stakeholder groups involved in the Habitat Hero System. Developers should use it to understand the required features, system behaviour, and external integrations such as the OneMap API and HDB data sources. Testers and QA engineers can rely on the functional and non-functional requirements to derive test cases and validate that the implemented system matches expected workflows. Project managers may reference the document to confirm scope coverage, track deliverables, and align acceptance criteria with what is specified. System administrators should focus on the parts describing operational responsibilities such as database updates, geospatial data maintenance, and system-level configuration. End users are not expected to read the document in full, but the use cases and high-level descriptions provide a clear overview of what the system does and what users can accomplish with it.

For reading order, it is recommended to begin with the Product Scope to understand the goals and boundaries of the system. Next, read the Functional Requirements to see what the system must do in detail, followed by the use case diagram and use case descriptions to understand the expected user and administrator workflows end-to-end. Finally, review the Non-Functional Requirements to understand constraints such as performance, reliability, security, and other quality expectations that apply across the system.

1.4 Product Scope

Habitat Hero System is a decision-support application that helps users shortlist and rank HDB housing options by combining housing constraints, livability preferences, and commuting needs. Users provide key constraints such as location preferences, flat type, and affordability-related requirements, and they also specify livability considerations that reflect what they value in a living environment. Where commuting is relevant, the system supports travel analysis for 2 commuters only by allowing users to define destinations and evaluate travel feasibility and fairness using public-transport routing information. Based on these inputs, the system generates a ranked list of recommendations together with supporting details that help users compare options efficiently and make more informed choices. In addition to user-facing functions, the system includes administrator-facing capabilities for maintaining geospatial datasets, retrieving and updating the database, and managing system configuration values that affect how recommendations are produced.

1.5 References

OneMap API Documentation (Singapore Land Authority): routing/travel-time and map services used by the system.

HDB Data Source / Dataset Documentation (e.g., HDB resale-related datasets): housing attributes used for filtering and scoring.

URA Planning / Zoning Data Documentation (if used): long-term planning or land-use overlays for risk/awareness signals.

2. Overall Description

2.1 Product Perspective

The Habitat Hero System is a new, self-contained decision-support web application. It originates from the need to protect inexperienced home buyers from poor housing choices by systematically evaluating critical "livability" flaws, such as excessive heat from the West Sun, noise pollution, and unequal commute burdens. Unlike traditional property portals (e.g., PropertyGuru, HDB Flat Portal) that act as large inventory databases requiring users to manually filter and evaluate active listings, Habitat Hero functions as a lifestyle-first recommender system. It differentiates itself through

algorithmic soft scoring via a Global Match Index, multi-commuter fairness evaluation, and deterministic geospatial analysis for environmental and future development risks.

While operating independently rather than as a component of a larger corporate software suite, its functionality relies heavily on real-time and scheduled integrations with external Singapore government data services. The system interfaces with the following external systems:

- OneMap API (Singapore Land Authority): For retrieving public transport travel times required for the Commute Fairness Score.
- HDB Resale Transaction API (Data.gov.sg): For retrieving historical transaction records to compute price efficiency baselines.
- URA Master Plan Datasets: For executing internal geospatial analysis to identify future development risks and environmental factors.

2.2 Product Functions

The Habitat Hero System is composed of five major functional groups that work together to translate user constraints and lifestyle preferences into a ranked list of housing recommendations. The major functions the product must perform include:

- **Hybrid User Profiling**
  - Allows users to define non-negotiable structural constraints, such as budget limits, preferred towns, and minimum lease duration.
  - Enables users to select livability factors (e.g., Solar Orientation, Acoustic Comfort) and toggle them as either strict requirements or weighted preferences.
  - Provides multi-commuter settings to input two destination addresses for joint household travel analysis.

- **Recommendation Engine Logic**
  - Executes a "Hard Filtering" process to immediately exclude HDB blocks that violate structural constraints or strict requirements.
  - Performs "Soft Scoring" by calculating attribute scores for the remaining blocks based on the user's weighted preferences.
  - Queries the OneMap Routing API to calculate a Commute Fairness Score and Total Commute Efficiency based on public transport travel times.
  - Aggregates all scores into a Global Match Index to generate and display a ranked list of property recommendations.

- **Geospatial Analysis Features**
  - Calculates building footprint polygons to identify and flag units with high West Sun heat exposure.
  - Generates 100-meter spatial proximity buffers around transport lines (MRT tracks and expressways) to determine noise risk.
  - Overlays URA Master Plan zoning data to alert users of future development risks, such as proximity to undeveloped Reserve Sites.

- **User Account Management**
  - Supports user registration and secure login authentication mechanisms.
  - Allows authenticated users to securely save their active Hybrid User Profiles to the database for persistence.

    - Automatically retrieves and populates previously saved preferences upon user login.

- **System Administration**
    - Provides secure, headless RESTful API endpoints for managing backend operations.
    - Automates and manually triggers the synchronization of external datasets (e.g., Data.gov.sg HDB Resale transactions) to maintain data currency.
    - Allows administrators to dynamically tune global algorithm weighting parameters (e.g., noise penalties) without system recompilation.
    - Maintains system integrity by enabling user access revocation and recording all administrative actions in an audit log.

2.3 User Classes and Characteristics

The Habitat Hero System serves two distinct user classes, differentiated by their goals, frequency of use, technical expertise, and system privileges.

- **Home Seekers (Primary User Class)**
    - **Description & Importance:** This is the most important user class for the product. The system is fundamentally designed to assist these inexperienced buyers who lack the expertise to evaluate complex livability flaws, such as noise pollution or solar heat gain. The entire user interface and recommendation engine must cater to their needs.
    - **Technical Expertise & Experience:** Low to medium. They do not require knowledge of real estate market analytics or geospatial data processing. They expect a highly usable, modern web interface with intuitive sliders and map visualizations.
    - **Frequency of Use:** High during their active housing search period, but use will likely cease once a home is successfully purchased.
    - **Sub-classes based on Privilege:**
        - Guest Users: Utilize the recommendation engine without creating an account; their profile data and search constraints persist only for the duration of the active browser session.
        - Registered Users: Authenticated users who possess basic privileges allowing them to permanently save their Hybrid User Profiles (structural constraints and weighted preferences) to the database for retrieval in future sessions.

- **System Administrators (Secondary User Class)**
    - **Description & Importance:** While less important to the frontend user experience, administrators are critical for maintaining backend system accuracy, data freshness, and operational integrity.
    - **Technical Expertise & Experience:** High. They must be comfortable interacting with headless RESTful API endpoints, formatting JSON payloads, and understanding database synchronization processes.
    - **Frequency of Use:** Low to medium (e.g., occasional algorithm tuning, monitoring automated daily data syncs).
    - **Privilege Level:** Highest. They hold authorized roles (verified via API Key or JWT) granting them exclusive access to trigger external data synchronization, adjust global algorithm weights (e.g., noise penalty multipliers), and revoke access for flagged user accounts.

2.4 Operating Environment

The Habitat Hero System is designed using a client-server architecture. The operating environments for both the user-facing client and the backend server are defined below:

- **Client-Side Operating Environment:**
  - Hardware Platforms: The system shall support access from desktop computers, laptops, smartphones, and tablets.
  - Display Constraints: The interface must be fully responsive and functionally usable across devices. It must support a minimum viewport width of 360 to 375 pixels for mobile devices and be optimized for full HD desktop displays up to 1920x1080 resolution.
  - Software & Browsers: As a browser-based web application, it requires no dedicated client-side installation. It must function without layout errors across the latest stable versions of modern web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.
- **Server-Side Operating Environment:**
  - Hardware Platforms: The application and its internal data storage will be hosted on a remote application server accessed over a network connection.
  - Operating System & Stack: The system shall be deployable on an operating system capable of hosting a web server and a database server. It must utilize a standard web application stack that supports HTTP/HTTPS communication, JSON data serialization, and session-based authentication.
- **External Software Coexistence:**
  - The backend server environment must peacefully coexist and maintain secure outbound communication (via HTTPS over TCP/IP) with external government services. Specifically, it must integrate with the Singapore Land Authority's OneMap Routing API and the official Data.gov.sg HDB Resale Transaction Dataset API.

2.5 Design and Implementation Constraints

The design and implementation of the Habitat Hero System are subject to several technical, regulatory, and architectural constraints that limit the options available to the developers:

- ***Regulatory and Compliance Policies:***
  - *The system must adhere to Singapore's Personal Data Protection Act (PDPA) regarding the handling of user email addresses and precise personal commute destinations.*
  - *To enforce the PDPA's Purpose Limitation and Retention Limitation principles, Personally Identifiable Information (PII) for guest users must exist strictly within the browser's local state and must be purged immediately upon session termination.*
  - *Developers must implement strict rate-limiting and local caching for static government datasets to act as a financial safeguard against unintended API overages from chargeable external services.*
- ***Performance and Timing Limitations:***
  - *The Recommendation Engine must calculate the Global Match Index and render the Results Dashboard within 3.0 seconds of the user submitting their profile.*
  - *The system must wait a maximum of 5.0 seconds for a response from the external OneMap Routing API; if it exceeds this, the system must invoke a graceful degradation exception rather than hanging.*

  - *The system must support up to 50 concurrent users interacting with the quiz simultaneously without degrading the response time beyond 3.0 seconds.*
- ***Hardware and Storage Limitations:***
  - *During automated data synchronization, the system must check available disk space before initiating a download of external datasets.*
  - *If insufficient storage is detected, the operation must abort and return an HTTP 507 Insufficient Storage error code.*
- ***Interfaces and Communication Protocols:***
  - *All client-to-server and outbound communications with external APIs (OneMap, Data.gov.sg) must use HTTPS over TCP/IP.*
  - *Data exchange with external services must be serialized using JSON format.*
- ***Security Considerations:***
  - *User passwords must never be stored in plaintext; developers must use a standard cryptographic hash function (e.g., bcrypt) for all account registrations.*
  - *All administrative API endpoints (/api/admin/*) must enforce strict authentication, requiring a valid Administrator API Key or JSON Web Token (JWT) in the HTTP Authorization header.*
  - *The system must prevent the exposure of sensitive information, such as password hashes and API keys, in client-side code.*
- ***Design Conventions and Architectural Standards:***
  - *The system architecture must enforce a strict separation of concerns between the User Interface (UI), Recommendation Engine, and Database Access layers.*
  - *Within the Recommendation Engine, the codebase for "Hard Filtering" must be structurally isolated from the "Soft Scoring" code to ensure maintainability, allowing developers to adjust algorithm weights without breaking the rest of the application.*

2.6 User Documentation

The Habitat Hero System is designed with an intuitive, self-guiding Single Page Application (SPA) web interface, which minimizes the need for extensive external documentation. The following user documentation components will be integrated and delivered directly within the software:

- ***Contextual Help and Tooltips:*** *The system shall provide embedded contextual tooltips and informational pop-overs accessible directly throughout the graphical user interface. This on-line help will assist users in understanding complex configuration inputs and livability factors (e.g., explaining how the "Acoustic Comfort" buffer is calculated) without requiring them to leave the active page.*

- ***Inline Validation and Error Notifications:*** *The system shall display clear, inline validation messages for input errors (e.g., when a user exceeds a budget limit or enters an invalid postal code) and non-blocking notifications for system-level errors. This guides users toward successful task completion and acts as an immediate troubleshooting guide.*

- ***Interactive Result Dashboards:*** *The recommendation results interface will feature self-explanatory environmental and future risk insight views (such as visual flags for West Sun exposure, noise risk, and zoning alerts). These dashboards inherently explain the scoring logic and trade-offs to the user visually.*

- ***Delivery Formats and Standards:*** *Due to the embedded, SPA-based nature of the interface and the descriptive design of the result dashboards, no external, printed user manuals, PDFs, or separate training tutorials are necessary or planned for delivery. All documentation and on-line help will be delivered in standard HTML5/CSS3 formats, natively integrated into the*

application's frontend components to ensure immediate accessibility across all supported devices.

2.7 Assumptions and Dependencies

Assumptions:

- Geospatial Data Accuracy: It is assumed that the building footprint polygons, transport network geometries, and URA Master Plan zoning boundaries retrieved from external datasets are physically accurate and up-to-date. The system relies deterministically on this data to calculate environmental scores (e.g., West Sun exposure, noise buffers).
- Client Device Capabilities: It is assumed that users will access the application using modern web browsers with JavaScript enabled. This is necessary to support the dynamic rendering of the Single Page Application (SPA) interface, interactive maps, and instantaneous UI updates without requiring full server round-trips.
- Continuous Funding and Access: It is assumed that the external government APIs currently utilized by the system will remain publicly accessible and free (or within acceptable usage tiers), and that their data usage policies will not change in a way that prohibits the system's core functions.

Dependencies:

- SLA OneMap API: The system’s Multi-Commuter Analysis and Commute Fairness Score are strictly dependent on the continued availability, responsiveness, and consistent JSON formatting of the Singapore Land Authority's OneMap Routing API to calculate accurate public transport travel times.
- Data.gov.sg HDB Resale API: The application depends on this external dataset as the single source of truth for historical pricing baselines and price efficiency calculations. Any downtime or deprecation of this API will temporarily degrade the system's financial filtering capabilities.
- URA Master Plan Overlays: The system depends on the availability of the Urban Redevelopment Authority's Master Plan datasets to successfully identify and overlay future development risks (e.g., undeveloped Reserve Sites) onto the interactive map.
- Third-Party Hosting: The application's operational uptime is dependent on the reliability of the chosen cloud infrastructure and hosting provider for both the web server and the PostgreSQL database server.

3. External Interface Requirements

3.1 User Interfaces

The Habitat Hero system provides two main user interfaces:

3.1.1 User Web Application Interface

- **EIR-1.1:** The system shall provide a responsive web-based graphical user interface supporting user registration, login, preference configuration, recommendation viewing, and block-level insights.

- **EIR-1.2:** The system shall provide user interface screens to support account management functions, including user registration and login.
- **EIR-1.3:** The system shall provide user interface screens for hybrid profile setup, including structural constraints, livability preferences, and multi-commuter inputs.
- **EIR-1.4:** The system shall provide a profile summary and confirmation screen prior to recommendation generation.
- **EIR-1.5:** The system shall provide a recommendation results interface displaying a ranked list of blocks and corresponding map view.
- **EIR-1.6:** The system shall provide environmental and future risk insight views, including west sun exposure, noise risk, and zoning alerts.
- **EIR-1.7:** The system shall provide contextual help and guidance accessible throughout the interface.
- **EIR-1.8:** The system shall provide a persistent navigation header across all pages.
- **EIR-1.9:** The system shall display non-blocking notifications for system-level errors where appropriate.
- **EIR-1.10:** The system shall display inline validation messages for input errors.

3.1.2 Administrator Web Portal Interface

- **EIR-1.11:** The graphical user interface (GUI) is exclusively designed for the 'User' (Home Seeker) actor. There is no frontend GUI for the System Administrator. Administrative tasks (data synchronization, configuration tuning, user revocation) are executed headlessly via secured backend REST API endpoints as defined in Section 4.5.

3.2 Hardware Interfaces

• **EIR-2.1:** The system shall operate as a browser-based web application hosted on a remote application server and accessed over a network connection.

• **EIR-2.2:** The system shall support access from desktop and laptop computers running on modern web browsers, such as Google Chrome and Microsoft Edge.

• **EIR-2.3:** The system shall support access from mobile devices, including smartphones and tablets, via responsive web design compatible with standard mobile browsers.

• **EIR-2.4:** The system shall support standard input and output peripherals, including keyboard, mouse, trackpad, and touchscreen interactions.

• **EIR-2.5:** The system shall not require direct integration with device sensors, such as GPS, for core functionality.

• **EIR-2.6:** The system shall require an internet connection to access external data sources and to communicate with backend servers via standard HTTP/HTTPS protocols.

• **EIR-2.7:** The system shall support responsive user interface rendering across a minimum viewport width of 375 pixels (mobile devices) up to 1920 pixels (full HD desktop displays), ensuring consistent usability and layout adaptability across varying screen sizes.

3.3 Software Interfaces

3.3.1 External APIs and Services

The system shall integrate with the following external software interfaces:

- **EIR-3.1:** The system shall integrate with the OneMap Routing API to retrieve public transport travel time between source and destination for commute fairness scoring.
- **EIR-3.2:** The system shall integrate with the official data.gov.sg HDB Resale Transaction Dataset API to retrieve the latest resale transaction history for baseline computation.
- **EIR-3.3:** The system shall retrieve geospatial datasets required for environmental and future development analysis to be used for footprint polygons, transport line geometries, and URA zoning overlays.

3.3.2 Internal Data Storage Interface

The system shall store application data in an internal database.

- **EIR-3.4:** The system shall store user information, including structural constraints and weighted preferences.
- **EIR-3.5:** The system shall store resale transaction records retrieved from external data sources.
- **EIR-3.6:** The system shall store computed price efficiency baseline values derived from resale transaction data.
- **EIR-3.7:** The system shall store global algorithm configuration parameters, such as noise penalty multiplier and west sun penalty scores.
- **EIR-3.8:** The system shall store audit log entries recording all administrative actions, including data synchronization and configuration changes.

3.3.3 Runtime Environment Interfaces

- **EIR-3.9:** The system shall be deployable on an operating system capable of hosting a web server and database server.

- **EIR-3.10:** The system shall utilize a standard web application stack that supports HTTP/HTTPS communication, JSON data serialization and session-based authentication.
- **EIR-3.11:** The system shall use standard libraries or frameworks for HTTP request handling, authentication, and geospatial computation.

3.4 Communications Interfaces

3.4.1 Network Protocols

- ***EIR-4.1:*** *The system shall use HTTPS over TCP/IP for all client-to-server communications.*
- ***EIR-4.2:*** *The system shall use HTTPS for all outbound communication with external APIs.*
- ***EIR-4.3:*** *The system shall maintain persistent and secure HTTP connections for data exchange between frontend and backend components.*

3.4.2 Message Formatting

- ***EIR-4.4:*** *The system shall exchange API requests and responses using JSON format where supported by external services.*
- ***EIR-4.5:*** *The system shall represent geographic coordinates using latitude and longitude decimal formats.*
- ***EIR-4.6:*** *The system shall standardize date or time formats in API payloads where applicable.*
- ***EIR-4.7:*** *The system shall validate incoming API responses to ensure required fields and data types are present before processing.*

3.4.3 Communication Security

- ***EIR-4.8:*** *The system shall encrypt all web traffic using Transport Layer Security (TLS) via HTTPS.*
- ***EIR-4.9:*** *The system shall require authentication tokens for administrator actions and shall reject unauthorized requests.*
- ***EIR-4.10:*** *The system shall prevent exposure of sensitive information, including password hashes, API keys or secret tokens in client-side code.*

3.4.4 Synchronization Mechanisms

- ***EIR-4.11:*** *The system shall support administrator-triggered synchronization for resale transaction updates.*

- ***EIR-4.12:*** *The system shall record the status of each synchronization process, including success or failure and timestamp.*
- ***EIR-4.13:*** *The system shall ensure that updated configuration parameters and recomputed baselines are applied to subsequent user requests without requiring system restart.*

4. System Features

4.1 Hybrid User Profiling

4.1.1 Description and Priority

The Hybrid User Profiling feature allows home seekers to define their housing search criteria by combining non-negotiable structural constraints (e.g., budget, flat type) with flexible, weighted lifestyle preferences (e.g., solar orientation, convenience). It also allows users to configure multi-commuter destinations to evaluate daily travel needs for up to two individuals.

4.1.2 Stimulus/Response Sequences

Stimulus: The user initiates the search process by navigating to the profile configuration module.

Response: The system displays the "Structural Constraints" form.

Stimulus: The user inputs Hard Filters including Budget Limit, Preferred Region, Flat Type, and Minimum Lease Remaining.

Response: The system temporarily stores these inputs in the active session.

Stimulus: The user enables the Multi-Commuter Analysis setting.

Response: The system dynamically adapts the UI to display input fields for destination addresses without page reloads.

Stimulus: The user inputs their destination addresses.

Response: The system validates the addresses and temporarily saves them.

Stimulus: The user selects a Livability Factor (e.g., Solar Orientation, Acoustic Comfort) and toggles it to a "Strict Requirement".

Response: The system records the factor as a Hard Constraint and hides the Priority Weight slider.

Stimulus: The user selects a Livability Factor and toggles it to a "Weighted Preference".

Response: The system dynamically enables and displays a Priority Weight slider for that specific factor.

Stimulus: The user adjusts the Priority Weight slider.

Response: The system records the decimal weight value.

Stimulus: The user submits the configuration.

Response: The system validates all inputs, saves the profile to the active session, and triggers the Recommendation Engine (UC-04).

4.1.3 Functional Requirements

REQ-1.1: When the user inputs Structural Constraints (maximum budget range, preferred towns, preferred flat types, and minimum remaining lease duration), the system shall store these values as Hard Constraints to filter the HDB block dataset.

REQ-1.2: If the user enters an invalid constraint (e.g., a negative value for the budget) and submits the form, the system shall display an error message highlighting the invalid field and halt the submission.

REQ-1.3: When the user selects a Livability Factor (Solar Orientation, Acoustic Comfort, or Convenience), the system shall allow the user to toggle its mode to either a "Strict Requirement" or a "Weighted Preference".

REQ-1.4: When the user designates a Livability Factor as a "Weighted Preference", the system require the user to assign a Priority Weight using a decimal value strictly greater than 0 and less than or equal to 1.

REQ-1.5: When the user enables the Multi-Commuter Analysis setting, the system shall allow the user to input a primary destination address (Destination A) and a secondary destination address (Destination B).

REQ-1.6: If the external map service fails to autocomplete commuter addresses during setup, the system shall allow manual text entry and display a warning stating "Precise commute calculation may be degraded.".

4.2 Recommendation Engine Logic

4.2.1 Description and Priority

The Recommendation Engine is the core computational algorithm of the system. It processes the user's saved profile to filter out unsuitable HDB blocks (Hard Filtering), evaluates the remaining blocks against lifestyle preferences (Soft Scoring), integrates external commute routing data, and generates a final ranked list of personalized housing matches.

Priority: High (Core Functionality).

4.2.2 Stimulus/Response Sequences

Stimulus: *The user submits a valid Hybrid User Profile from the configuration module.*

Response: *The system executes the Hard Filtering process to exclude invalid blocks from the dataset.*

Response: *The system executes the Soft Scoring process and queries the external OneMap Routing API for travel times (if commuter destinations are defined).*

Response: *The system calculates the Global Match Index, ranks the properties, and displays the "Results Dashboard".*

Alternative Sequence (No Matches): *If no blocks pass the Hard Filtering stage, the system displays a "No Matches" page suggesting the user relax their constraints.*

Exception Sequence (API Timeout): *If the OneMap API times out, the system skips commute scoring, ranks based only on lifestyle factors and displays the results with a warning message.*

4.2.3 Functional Requirements

REQ-2.1 (Hard Filtering): When the recommendation engine is triggered, the system shall query the internal database to exclude any HDB blocks that violate the user's defined Structural Constraints (e.g., Budget), outputting a valid candidate list of blocks.

REQ-2.2 (Zero Matches Exception): If the Hard Filtering process results in zero candidate blocks, the system shall halt the scoring process and output a prompt suggesting the user to relax structural constraints ("No Matches Found. Please broaden your search criteria.").

REQ-2.3 (Soft Scoring): For each block in the candidate list, the system shall calculate an Attribute Score (0-1.0) for each active preference and multiply it by the user's assigned Priority Weight to prepare for aggregation.

REQ-2.4 (Commute Fairness Calculation): If the user profile contains two destination addresses, the system shall query the OneMap Routing API to retrieve public transport travel times (T_A and T_B) and calculate a Commute Fairness Score based on the absolute difference (|T_A - T_B|), utilizing a +1 smoothing factor to prevent division by zero errors.

REQ-2.5 (Total Commute Efficiency Filter): When the user defines a maximum threshold for Total Travel Time, the system shall exclude any candidate block where the calculated combined travel time (T_A + T_B) exceeds the specified threshold.

REQ-2.6 (External API Timeout Handling): If the OneMap Routing API fails to respond within 5 seconds during the commute calculation phase, the system shall set the commuteFairnessScore and totalCommuteBurden attributes to null, complete the Global Match Index ranking using only the remaining active Soft Constraints, and output the results with a warning banner stating "Commute data temporarily unavailable".

REQ-2.7 (Global Match Index Generation): After evaluating all constraints and scores, the system shall aggregate the results into a Global Match Index (0-100%) for each block, sort the blocks in descending order, and display the ranked list on the Results Dashboard.

4.3 Geospatial Analysis Features

4.3.1 Description and Priority

The Geospatial Analysis module is a background processing service that enriches HDB block data with environmental attributes. It utilizes building footprint polygons, transport line geometries, and urban planning overlays to deterministically calculate risks regarding solar heat gain, noise pollution, and future development obstructions.

Priority: Medium (Critical Support Feature).

4.3.2 Stimulus/Response Sequences

Stimulus: The System Administrator sends an authenticated request to trigger the "Maintain Geospatial Data" operation (or a scheduled weekly background cron job initiates).

Response: The system retrieves the latest "Building Footprints", "Transport Lines", and "URA Master Plan" geographic datasets from the internal database.

Response: The system iterates through all registered HDB blocks and calculates the façade azimuth for each using the footprint polygons.

Response: The system updates the westSunStatus attribute to "True" for any block with an azimuth between 260° and 280°.

Response: The system generates 100-meter spatial buffers around the retrieved transport line geometries.

Response: The system flags any HDB block whose coordinates fall within these buffers by setting its noiseRiskLevel to "High".

Response: The system performs a spatial intersection check against the URA Master Plan layers and updates the futureRiskFlag for blocks within 100m of a "Reserve Site".

Response: The system saves all updated attributes to the database and returns a successful confirmation log.

4.3.3 Functional Requirements

REQ-3.1 (Solar Orientation Calculation): The system shall analyze the geometry of each HDB block's building footprint polygon to calculate the azimuth angle of its main facade.

REQ-3.2 (West Sun Flagging): If the calculated azimuth angle falls within the range of 260° to 280°, the system shall set the block's westSunStatus to "True" (High Heat Exposure).

REQ-3.3 (Noise Buffer Generation): The system shall generate a 100-meter spatial buffer zone around all polyline geometries classified as "Above-Ground MRT Tracks" or "Expressways".

REQ-3.4 (Noise Risk Classification): The system shall identify any HDB block whose centroid coordinates fall within the generated noise buffer and update its noiseRiskLevel to "High".

REQ-3.5 (Future Risk Identification): The system shall calculate the straight-line distance between the coordinates of the HDB block and the center coordinates of any land zoned as "Reserve Site". If the distance is less than 100 meters, the system shall set the futureRiskFlag to True.

REQ-3.6 (Missing Geometry Exception): If a block lacks valid building footprint data (preventing azimuth calculation), the system shall set its environmental attributes to "Unknown" and exclude it from strict "West Sun" filtering to prevent false negatives.

4.4 Recommendation Engine Logic

4.4.1 Description and Priority

The User Account Management feature handles user registration, secure authentication, and the persistence of customized user preferences. It allows users to save Hybrid User Profiles (Structural Constraints and Weighted Preferences) for future sessions, enhancing convenience and personalization.

Priority: High.

4.4.2 Stimulus/Response Sequences

Stimulus: An unregistered user submits a valid email and password on the Registration page.

Response: The system encrypts the password, creates the account, automatically logs the user in, and displays a success message.

Alternative Sequence (Duplicate Email): If the email already exists, the system rejects the registration and prompts the user to log in instead.

Stimulus: A registered user submits their email and password on the Log In page.

Response: The system validates the credentials, generates a session token, and redirects the user to the Dashboard while loading any saved preferences.

Exception Sequence (Account Lockout): If the user enters incorrect credentials 5 consecutive times, the system temporarily locks the account for 15 minutes.

Stimulus: A logged-in user clicks "Save Profile".

Response: The system persists the active Hybrid User Profile to the database.

4.4.3 Functional Requirements

REQ-4.1 (Account Creation): When an unregistered user submits a valid email address and password, the system shall hash the password, save the new account record to the database, and automatically authenticate the user.

REQ-4.2 (Duplicate Account Handling): If a user attempts to register with an email address already linked to an existing account, the system shall reject the submission, display the message "Registration Invalid. Account already exists", and provide a hyperlink to the Log In page.

REQ-4.3 (Weak Password Rejection): If a user submits a password during registration that fails the system's security policy (e.g., length or complexity), the system shall reject the input and prompt the user for a stronger password.

REQ-4.4 (User Authentication): When a user submits an email and password on the Log In page, the system shall verify the credentials against the database, generate a session token (expiring after 24 hours of inactivity), and redirect the user to the application.

REQ-4.5 (Invalid Credentials Handling): If a user submits an incorrect email or password combination, the system shall clear the password input field and display the error message "Invalid email or password".

REQ-4.6 (Account Lockout Security): If a user enters an incorrect password five (5) consecutive times, the system shall temporarily lock the account for 15 minutes and display the message "Too many attempts. Please try again later".

REQ-4.7 (Profile Persistence): When an authenticated user triggers a save action, the system shall permanently save their current SavedProfile (containing the StructuralConstraint, CommuterProfile, and SoftConstraint entities) to the database, linked to their account ID.

REQ-4.8 (Profile Retrieval): Upon a successful user login, the system shall query the database for a previously saved Hybrid User Profile and automatically populate the search configuration inputs with these retrieved values.

4.5 System Administration

4.5.1 Description and Priority

The System Administration feature provides authorized personnel with a secure, headless mechanism to manage the platform's backend operations via RESTful API endpoints. It handles the synchronization of external datasets (HDB Resale Transactions, Geospatial geometries), allows the live tuning of global recommendation algorithm weights, and maintains system integrity through user access revocation and audit logging.

Priority: Medium (Critical for maintenance and accuracy).

4.5.2 Stimulus/Response Sequences

Stimulus: The Administrator sends an authenticated POST request to the data synchronization endpoint (e.g., /api/admin/sync/hdb).

Response: The system validates the API Key/Bearer Token in the request header.

Response: The system connects to the external Data.gov.sg API, downloads the dataset, and updates the internal database.

Response: The system returns an HTTP 200 OK JSON response containing the message "Database Updated" and the new timestamp.

Exception Sequence (Connection Failure): If the external API is unreachable, the system returns an HTTP 502 Bad Gateway response with error details, after attempting to reconnect for 30 seconds.

Stimulus: The Administrator sends a PATCH request to the configuration endpoint with a new JSON payload (e.g., {"west_sun_penalty": 0.8}).

Response: The system updates the active runtime variables and returns an HTTP 200 OK confirmation.

Stimulus: The Administrator sends a DELETE request to the user management endpoint (e.g., /api/admin/users/{userId}).

Response: The system revokes the user's session and returns an HTTP 204 No Content confirmation.

4.5.3 Functional Requirements

REQ-5.1 (Automated Data Synchronization): The system shall automatically trigger a background job to synchronize with the Data.gov.sg HDB Resale Price API at least once every 24 hours to ensure data freshness.

REQ-5.2 (API Authentication): The system shall enforce strict authentication on all /api/admin/* endpoints, requiring a valid Administrator API Key or JSON Web Token (JWT) in the HTTP Authorization header for every request.

REQ-5.3 (Data Ingestion & Override): Upon receiving a valid synchronization request, the system shall parse the retrieved external data, overwrite the existing internal database records, and update the "Price Efficiency" baseline without disrupting active user sessions.

REQ-5.4 (Connection Error Handling): If the system fails to connect to an external database during a sync operation, it shall log the error internally and return a structured JSON error response (e.g., HTTP 502) rather than crashing or hanging .

REQ-5.5 (Storage Safety Check): The system shall check available disk space before initiating a data download; if insufficient space is detected, it shall abort the operation and return an HTTP 507 Insufficient Storage error code.

REQ-5.6 (Dynamic Algorithm Tuning): When the system receives a valid configuration update request, it shall immediately apply the new global weighting variables to the active recommendation engine instance, affecting all subsequent search requests.

REQ-5.7 (User Access Revocation): Upon receiving a valid account revocation request, the system shall instantly invalidate the targeted user’s session token and flag the account status as "Suspended" in the database.

REQ-5.8 (Audit Logging): The system shall record every request made to an administrative endpoint, including the Admin ID, Timestamp, Endpoint URL, and IP Address, into a secure, immutable audit log file.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-1.1 (Recommendation Engine Latency): Under normal operating conditions, the system shall calculate the Global Match Index and render the Results Dashboard within 3.0 seconds of the user submitting the Hybrid User Profile.

- Rationale: Users expect near-instant feedback from modern web applications. Keeping the calculation under 3 seconds prevents user frustration and perceived system freezing.
-

NFR-1.2 (External API Timeout Bounds): The system shall wait a maximum of 5.0 seconds for a response from the OneMap Routing API before invoking the graceful degradation exception.

- Rationale: External government APIs can occasionally experience high traffic or downtime. A strict 5.0-second cutoff ensures the application does not hang indefinitely and can still deliver a partial result during a live demonstration.

NFR-1.3 (UI Responsiveness): The screen shall update instantly (in under 1 second) when the user interacts with it, such as clicking buttons or moving sliders, without requiring the web page to refresh.

- Rationale: To provide a seamless, modern Single Page Application (SPA) experience, UI components must react immediately to input without the latency of a full server round-trip.

NFR-1.4 (Concurrent Load): The system shall support a minimum of 50 concurrent users generating recommendations simultaneously without degrading the response time beyond 3.0 seconds.

- Rationale: This baseline accommodates the expected peak load during a classroom peer-review or grading session, ensuring the application remains stable when multiple evaluators test it at the exact same time.

5.2 Safety Requirements

NFR-2.1 (Financial Liability Disclaimer): Because the system acts as a decision-support tool for high-value real estate transactions, all recommendation dashboards shall prominently display a disclaimer stating that the matches are for informational purposes only and do not constitute official real estate or financial advice. This safeguard prevents potential legal or financial harm resulting from user reliance on the algorithm.

NFR-2.2 (Financial Safeguard against API Overages): To prevent unintended financial loss due to chargeable external services (e.g., OneMap Routing API high-tier usage), the system shall implement strict rate-limiting and local caching for static government datasets. This enforces the course policy stating that the team must manage API usage effectively to avoid excess charges.

Safety Certifications & Regulations: As a purely informational web application, the system does not interface with physical hardware and therefore requires no physical safety certifications.

5.3 Security Requirements

NFR-3.1 (Data in Transit): All data transmitted between the user's browser and the application server shall be encrypted using standard HTTPS protocols.

NFR-3.2 (Cryptographic Hash Standard): User passwords shall never be stored in plaintext. The system shall utilize a standard cryptographic hash function (e.g., bcrypt) for all account registrations to ensure credential security.

NFR-3.3 (Session-Based PII Lifecycle): For guest users, personally identifiable information (such as exact workplace addresses entered for commute calculations) shall exist strictly within the browser's local state and shall be immediately purged upon session termination.

Compliance with External Policies (PDPA): The handling of user email addresses and personal commute destinations shall adhere to the core principles of Singapore's Personal Data Protection Act (PDPA), specifically the obligations of Purpose Limitation and Retention Limitation (as enforced by NFR-3.3).

Security Certifications: Given the academic scope of this prototype, formal enterprise security certifications (e.g., ISO 27001) are not required.

5.4 Software Quality Attributes

Attribute Preferences: For this project, Usability and Robustness are prioritized over Adaptability. Because the goal is to demonstrate a working prototype, it is more important that the interface is highly intuitive (Usability) and does not crash during external API failures (Robustness) than it is to build an architecture that can adapt to other countries' real estate markets (Adaptability).

NFR-4.1 (Robustness): The system shall handle external errors without crashing. If the OneMap API fails to respond within the 5.0-second timeout window, the system shall catch the error and continue to rank properties using only the internal user constraints.

NFR-4.2 (Usability): The web application interface shall be optimized for standard laptop displays (1920x1080 resolution) so that all sliders and map results are fully visible without horizontal scrolling.

NFR-4.3 (Maintainability): The Recommendation Engine's code shall be separated into distinct parts. The "Hard Filtering" code shall be isolated from the "Soft Scoring" code so developers can easily adjust algorithm weights without breaking the rest of the system.

NFR-4.4 (Testability): The recommendation algorithm shall produce consistent results. When provided with the exact same user inputs against the same database, the system must output the exact same match scores 100% of the time.

5.5 Business Rules

BR-1 (User Privileges): Only registered and authenticated users are permitted to save Hybrid User Profiles to the database. Guest users may utilize the recommendation engine but their data shall only persist for the duration of the active browser session.

BR-2 (Data Authority): The Data.gov.sg HDB Resale API serves as the single source of truth for "Price Efficiency" calculations. Historical transaction data older than 24 months shall be automatically excluded or heavily penalized in efficiency calculations to ensure relevant market accuracy.

BR-3 (Administrative Access): Only personnel with the "System Administrator" role authorization are permitted to invoke the dynamic tuning of global algorithm weights or manually trigger external data synchronization.

6. Other Requirements

6.1 Database Requirements

OR-1.1: The system shall use a relational database to store user accounts, saved Hybrid User Profiles, resale transactions, geospatial attributes, configuration parameters, and audit logs.

OR-1.2: The system shall store geospatial data using spatial data types supported by the database engine to enable efficient buffer, intersection, and centroid calculations.

OR-1.3: The system shall enforce referential integrity between user accounts and saved profiles using foreign key constraints (Every hybrid profile shall be linked to a User account between the respective tables within the internal database).

OR-1.4: The system shall index frequently queried fields, including block ID, town, price range, and lease duration, to support the 3.0 second performance target defined in NFR-1.1.

OR-1.5: The system shall maintain separate database tables for Hard Constraints, Weighted Preferences, and computed scores to preserve data normalisation and maintainability.

6.2 Data Retention and Archival Requirements

OR-2.1: The system shall automatically remove guest session data upon browser session termination in compliance with NFR-3.3.

OR-2.2: The system shall retain resale transaction records only as long as required for baseline computation and shall archive or purge outdated raw datasets beyond 24 months if no longer required for computation.

6.3 Logging and Monitoring Requirements

OR-3.1 (Error Logging): The system shall log all critical runtime errors, including API timeouts, database failures, and synchronisation failures, with timestamp and severity level.

OR-3.2 (Performance Logging): The system shall log performance metrics for each recommendation request, including total computation time and external API response time, to verify compliance with NFR-1.1 and NFR-1.2.

OR-3.3 (Admin Monitoring): The system shall provide administrators with access to system health indicators, including last successful data synchronisation timestamp and current database size.

6.4 Legal and Compliance Requirements

OR-4.1: The system shall comply with the Singapore Personal Data Protection Act under the oversight of the Personal Data Protection Commission.

OR-4.2: The system shall display a Privacy Policy page explaining what personal data is collected, the purpose of collection, retention duration, and user rights.

OR-4.3: The system shall display Terms of Use stating that the platform is an academic prototype and does not replace official services provided by the Housing and Development Board.

OR-4.4 (Dataset Citations): The system shall attribute external data sources where required, including data obtained from Data.gov.sg and the Singapore Land Authority.

6.5 Internationalisation and Localisation Requirements

OR-5.1 (Currency Standardisation): The system shall standardise all currency values in Singapore Dollars and display them using consistent formatting.

OR-5.2 (Date Standardisation): The system shall standardise date formats across the interface and API payloads to prevent ambiguity in transaction records.

6.6 Deployment and Environment Requirements

OR-6.1: The system shall support deployment in a containerised environment to ensure consistent setup across development, testing, and production environments.

OR-6.2: The system shall maintain separate configurations for development and production environments, including different API keys and database credentials.

OR-6.3: The system shall support automated build and deployment scripts to reduce manual configuration errors.

7. Appendix A: Data Dictionary

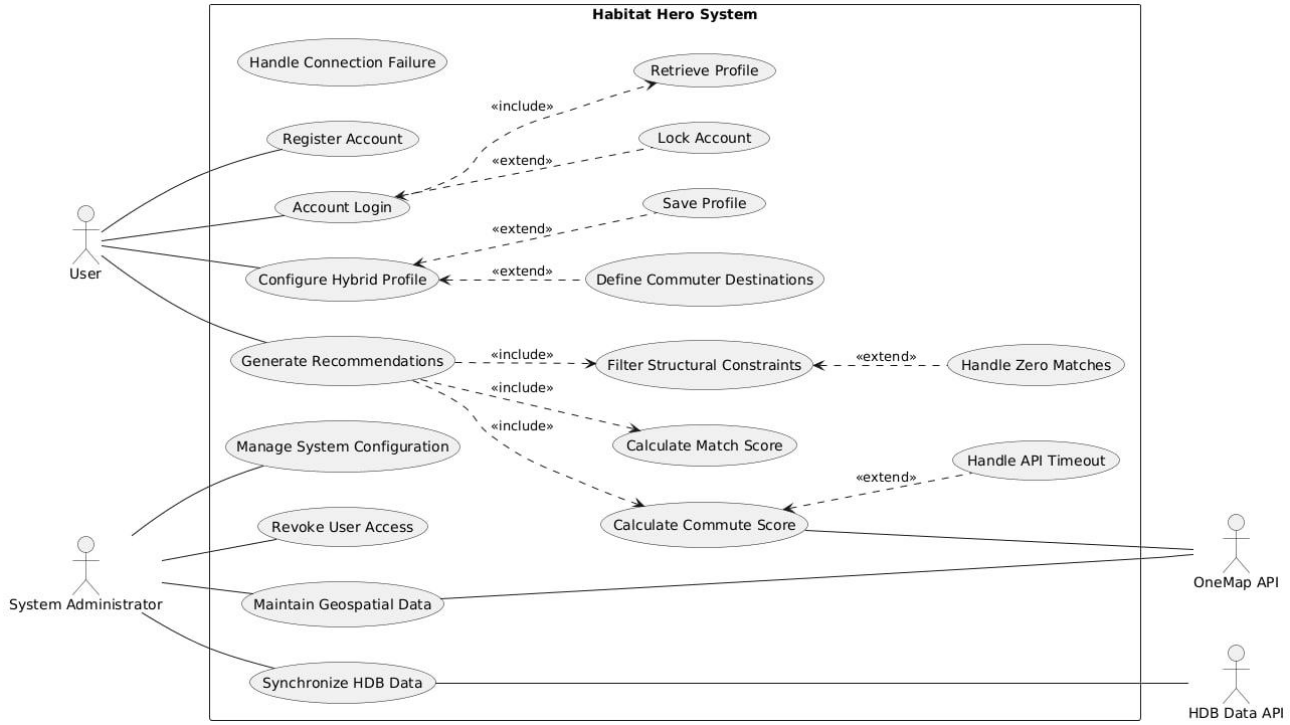
Term	***Type***	***Definition***	***Attributes / Data Elements***	***Relationships***	***Constraints / Notes***
UserAccount	*Entity*	*An entity representing a registered user, containing authentication credentials.*	*userID (Integer)* *email (String)* *passwordHash (String)* *isActive (Boolean)*	*1 UserAccount owns 0..* SavedProfiles*	*email must be unique and a valid format.* *passwordHash cannot be null.*
SavedProfile	*Entity*	*A stored record of a user's configuration, containing their constraints and factors.*	*profileID (Integer)* *lastUpdated (DateTime)*	*Each SavedProfile belongs to exactly 1 UserAccount.* *1 SavedProfile contains 1 StructuralConstraint, 1 CommuterProfile, and 0..* SoftConstraints.*	*-*
StructuralConstraint	*Entity*	*A non-negotiable search parameter that acts as a Hard Constraint filtering mechanism.*	*maxBudget (Decimal)* *preferredTowns (List<String>)* *preferredFlatType (String)* *minLeaseYears (Integer)*	*Belongs to exactly 1 SavedProfile.*	*maxBudget > 0* *minLeaseYears ≥ 0*
SoftConstraint	*Entity*	*A preference factor that influences ranking. Properties are penalized, not excluded, if failed.*	*factorName (String)* *priorityWeight (Decimal)* *isStrict (Boolean)*	*Belongs to exactly 1 SavedProfile.*	*If isStrict = true, this factor acts as a Hard Constraint.*
CommuterProfile	*Entity*	*A sub-entity of the user profile containing destination addresses for*	*destinationA (Coordinates)* *destinationB (Coordinates)*	*Belongs to exactly 1 SavedProfile.*	*destinationA and destination B are required if isEnabled = true.*

		commute logic.	*isEnabled (Boolean)*		
HDBBlock	`	*A physical residential building entity containing multiple units.*	*blockId (Integer)* *postalCode (String)* *town (String)* *coordinates (Coordinates)* *westSunStatus (Boolean)* *noiseRiskLevel (String)* *futureRiskFlag (Boolean)*	*1 HDBBlock has 0..* ResaleTransactions.* *1 HDBBlock is evaluated against 0..* SavedProfile.*	*coordinates latitude ∈ [-90,90], longitude ∈ [-180,180]*
ResaleTransaction	*Entity*	*A historical record of a flat sold, used for Price Efficiency analysis.*	*transactionId (Integer)* *resalePrice (Decimal)* *floorArea (Decimal)* *leaseCommencement (Date)*	*Each ResaleTransaction belongs to exactly 1 HDBBlock.*	*resalePrice > 0* *floorArea > 0*
priorityWeight	*Attribute*	*A multiplier applied to a Soft Constraint's score, representing subjective importance.*	*-*	*Belongs to SoftConstraint*	*0 < priorityWeight ≤ 1*
attributeScore	*Attribute*	*An objective score representing how well a property meets a specific criterion.*	*-*	*Belongs to HDBBlock (Evaluated State)*	*0 ≤ attributeScore ≤ 1.0*
globalMatchIndex	*Attribute*	*The final composite score used to rank properties*	*-*	*Belongs to HDBBlock (Evaluated State)*	*0 ≤ globalMatchIndex ≤ 100*

		(aggregation of all factors).			
westSunStatus	*Attribute*	*Indicates if a unit's main facade faces an azimuth angle resulting in high heat.*	*-*	*Belongs to HDBBlock*	*True if facade azimuth is 260°–280°*
noiseRiskLevel	*Attribute*	*Indicates if a block falls within a calculated spatial buffer around noise sources.*	*-*	*Belongs to HDBBlock*	*e.g., "High" if within 100m of above-ground MRT.*
futureRiskFlag	*Attribute*	*A warning flag generated if a block is located within a specific straight-line distance of an empty plot zoned as a "Reserve Site".*	*-*	*Belongs to HDBBlock*	*True if the block's coordinates are within 100m of a risky URA zone centroid.*
commuteFairnessScore	*Attribute*	*A calculated metric representing the equity of travel times.*	*-*	*Belongs to HDBBlock (Evaluated State)*	*0.0 to 1.0 (1.0 = Perfect Fairness)* *Can be null if routing API times out.*
totalCommuteBurden	*Attribute*	*The sum of travel times for both commuters ($T_A + T_B$) from a specific block.*	*-*	*Belongs to HDBBlock (Evaluated State)*	*≥ 0 minutes* *Can be null if routing API times out.*
SessionState	*Concept*	*The temporary storage of the user's current inputs while navigating the app before saving.*	*-*	*-*	*Temporary; cleared on logout/timeout.*

8. Appendix B: Analysis Models

Use Case Diagram



Class Diagram

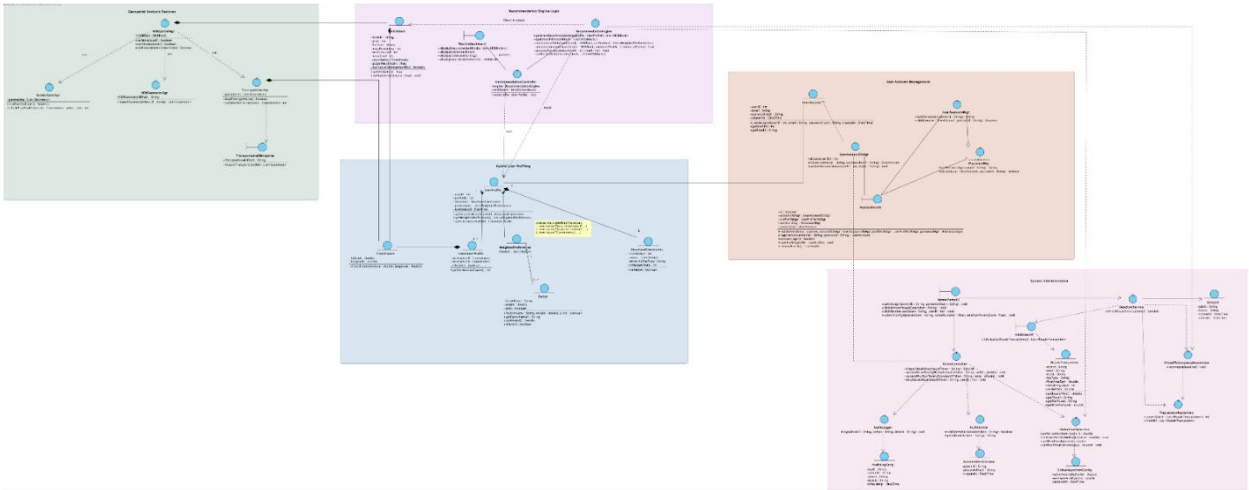
https://drive.google.com/file/d/1DTMSBOD6aXhiqnNN92sCJEp5ToGP3-N7/view?usp=sharing

State-Transition Diagram

https://drive.google.com/file/d/1-vlZqpnbcAFGtfouAJCbZEziit-zzTb_/view?usp=drive_link

Entity-Boundary-Control Class Diagram

https://drive.google.com/file/d/1ncqjl4mhHJDQZ5xUgIuBGWF2Jj9QEUDK/view?usp=sharing



9. Appendix C: Use Case Description

Use Case ID:	UC-01		
Use Case Name:	Register Account		
Created By:	Habitat Hero Team	Last Updated By:	
Date Created:	05 Feb 2026	Date Last Updated:	24 Feb 2026

Actor:	User
Description:	Allows a new user to create a personal account to enable the saving of preferences and Hybrid Profiles.
Preconditions:	1. The system is up and running. 2. The user has a valid internet connection. 3. Users can access the System. 4. The user is not currently logged in.

	5. The user has a valid email address.
Postconditions:	1. A new user account is created in the database. 2. The user is automatically authenticated(logged in). 3. Saved profile features are added
Priority:	High
Frequency of Use:	Low (Once per user)
Flow of Events:	The user clicks the “Sign Up” button. 1. System display Registration page. 2. User enters an email in the email field. 3. User enters a password in the password field. 4. User presses the register button. 5. System checks database no duplicate account. 6. System hashes the password and saves the new account record to database. 7. System logs the user in and displays a “Registration Success” message.
Alternative Flows:	**UC-01.AC.1: Email Already Exists** 1. System detects the email is linked to another account in the database. 2. System display "Registration Invalid. Account already exists” 3. System provides a link to Log In (UC-02) page. The user clicks a link to switch to log in.
Exceptions:	**UC-01.EX.1: Weak Password** 1. Password fails security policy (minimum 8 characters, containing at least 1 number and 1 special character) 2. The system rejects input and prompts for a stronger password.

Includes:	None
Special Requirements:	Passwords must be encrypted (hashed) before storage.
Assumptions:	None
Notes and Issues:	None

Use Case ID:	UC-02		
Use Case Name:	Log In		
Created By:	Habitat Hero Team	Last Updated By:	
Date Created:	05 Feb 2026	Date Last Updated:	05 Feb 2026

Actor:	User
Description:	Allows an existing user to authenticate their identity to access their saved preferences and history.
Preconditions:	1. The system is up and running. 2. The user has a valid internet connection. 3. Users can access the System. 4. The user has an existing account. 5. The user is currently not logged in.
Postconditions:	1. User is authenticated 2. System retrieves the user's previously saved preferences (if any).
Priority:	Medium
Frequency of Use:	Medium (Once per session)
Flow of Events:	User clicks the “Log In” button. 1. System display the login page. 2. User enters registered email in the email field. 3. User enters a password in the password field. 4. User presses the login button.

	5. System verifies the credentials against the database. 6. System generates a session token. The system redirects users to the Dashboard/Home with their saved settings loaded.
Alternative Flows:	**UC-02.AC.1: Invalid Credentials** 1. Password does not match the stored hash. 2. The system displays "Invalid email or password." 3. The system clears the password field. 4. User retries.
Exceptions:	**UC-02.EX.1: Account Locked** 1. The user enters the wrong password 5 times consecutively. 2. The system temporarily locks the account for 15 minutes. 3. The system displays "Too many attempts. Please try again later."
Includes:	None
Special Requirements:	Session tokens must expire after a set period of inactivity (e.g., 24 hours)
Assumptions:	Users remember their credentials (Forgot Password is out of scope for this project).
Notes and Issues:	None

Use Case ID:	UC-03		
Use Case Name:	Configure Hybrid Profile		
Created By:	Habitat Hero Team	Last Updated By:	
Date Created:	05 Feb 2026	Date Last Updated:	24 Feb 2026

Actor:	User
Description:	The user defines their housing search criteria, including financial limits (Hard Constraints), lifestyle needs (Soft Weights), and optional commuter destinations (for 1 or 2 people).
Preconditions:	1. System is up and running 2. User has valid internet connection 3. User has a valid account 4. User is logged In
Postconditions:	1. A valid Hybrid Profile containing all constraints and preferences are saved to weights are saved to the System database.

Priority:	High (Core Functionality)
Frequency of Use:	High (Every search session)
Flow of Events:	1. The user initiates the search process. 2. The system displays the "Structural Constraints" form. 3. User inputs Hard Filters: Budget Limit (Max Ceiling), Preferred Region, and Flat Type. 4. User defines Commuter Profile(s): a. User inputs 0 addresses (skips commute logic). b. User inputs 1 address (Standard Commute). c. User inputs 2 addresses (Fairness Commute). 5. The system displays the Livability Factors list (e.g., Nature, Food, Quiet). 6. The user selects preferences and adjusts the Priority Weight Slider (0-10) for each. 7. User toggles specific factors to "Strict Requirement" mode (e.g., "Strictly No West Sun"). 8. The user submits the configuration. 9. The system validates inputs and calls calls UC-04 (Generate Recommendations).
Alternative Flows:	**UC-03.AC.1: Validation Failure** 1. The user enters an invalid Budget (e.g., negative value) or unrecognized address. 2. The system displays an error message highlighting the invalid field. 3. The user corrects the input. 4. Use case resumes from Step 8.
Exceptions:	**UC-03.EX.1: Address Service Unavailable** 1. External map service fails to autocomplete addresses in Step 4. 2. The system allows manual text entry. 3. The system displays a warning: "Precise commute calculation may be degraded."
Includes:	UC-04 (Generate Recommendations)
Special Requirements:	The Commuter Profile input UI must dynamically adapt to accept 0, 1, or 2 input fields without page reloads.
Assumptions:	The user is accessing the system via a supported web browser with JavaScript enabled.
Notes and Issues:	None

Use Case ID:	UC-04		
Use Case Name:	Generate Recommendations		
Created By:	Habitat Hero Team	Last Updated By:	
Date Created:	05 Feb 2026	Date Last Updated:	24 Feb 2026

Actor:	User

Description:	The system processes the user's profile to filter out unsuitable properties and rank the remaining blocks based on a weighted "Global Match Index" (Lifestyle + Commute).
Preconditions:	1. System is up and running. 2. User has a valid internet connection. 3. User has a valid hybrid profile. 4. A valid User Profile exists (passed from UC-01). 5. Database connectivity is established.
Postconditions:	1. The user is presented with a prioritized list of HDB blocks. 2. The interactive map is loaded with markers.
Priority:	High
Frequency of Use:	High (Triggered immediately after UC-01)
Flow of Events:	1. System retrieves the active User Profile. 2. System queries database to exclude blocks that violate Structural Constraints (Budget, Flat Type) or Strict Requirements (e.g., West Facing). 3. System iterates through remaining blocks: a. Calculates attribute scores for active preferences. b. Applies user's Priority Weights. 4. System checks number of destination addresses: a. If 0: Skips commute scoring. b. If 1: Calls OneMap API for travel time (T_A); Score based on proximity. c. If 2: Calls OneMap API for both (T_A, T_B); Score = Fairness Index (1 / |T_A - T_B|) to Score = Fairness Index (1 - (|T_A - T_B| / (T_A + T_B + 1))). 5. System computes Global Match Index (Sum of weighted scores). 6. The system ranks properties from highest to lowest Index. 7. The system displays the Results Dashboard.
Alternative Flows:	**UC-02.AC.1: No Results Found** 1. Hard Filters are too strict (e.g., Budget too low for Region). 2. The system returns 0 candidates. 3. The system displays a 'No Matches Found. Please broaden your search criteria.'
Exceptions:	**UC-02.EX.1: OneMap Routing Timeout** 1. OneMap API does not respond within 5 seconds during Step 4.

	2. System skips Commute Score calculation. 3. The system completes the ranking using only Lifestyle Factors. 4. The system displays results with a warning: "Commute data temporarily unavailable."
Includes:	Hard Filtering, Soft Scoring, Commute Score Calculation
Special Requirements:	Commute Fairness calculation must handle edge cases (e.g., T_A = T_B) to avoid division by zero error (add +1 smoothing).
Assumptions:	Pre-calculated geospatial data (West Sun, Noise Buffers) is already available in the database (static lookup).
Notes and Issues:	None

Use Case ID:	UC-05		
Use Case Name:	Retrieve & Update Database		
Created By:	Habitat Hero Team	Last Updated By:	
Date Created:	08 Feb 2026	Date Last Updated:	20 Feb 2026

Actor:	System Administrator
Description:	The System Administrator initiates the update of System database through retrieval from External Databases
Preconditions:	1. System is up and running. 2. Internal Database connectivity is established. 3. External Databases connectivity is established
Postconditions:	An updated System database(Resale Transaction History, Building Footprint Polygon, Transport Line Geometry
Priority:	Low
Frequency of Use:	Medium (Non-functional requirement)
Flow of Events:	1. System Administrator sends an authenticated API request to the specific database update endpoint (e.g., /api/admin/update-hdb). 2. System establishes a connection with the source of data from the External Database. 3. Upon successful connection, System checks the currency of the External Database data against the internal database. 4. System downloads updated data from the External Database.

	5. System overrides existing data and updates the Internal Database. 6. System returns a JSON success response containing a confirmation message (e.g., "Database Updated") and the new database currency date.
Alternative Flows:	
Exceptions:	**UC-05.EX.1: External Database Connection Failure** 1. System displays exception message “Connection Failed” 2. System attempts to reestablish connection for 30 seconds a. If no response from external database, terminate updating of System Database b. If response received from external database, resume update from step 3 **UC-05.EX.2: External Database Data Out of Currency** 1. System displays exception message “External Database Data out of Currency” 2. System prompts “continue” and confirms the continuation of System database update a. If System Administrator select “Yes”, resume System Database update from step 4 b. If System Administrator select “No”, terminate System Database update. **UC-05.EX.3: Internal Database Insufficient Storage** 1. System terminates the download and database update 2. System display error message “Internal System Database out of storage”.
Includes:	None
Special Requirements:	
Assumptions:	

Notes and Issues:	None

10. Appendix D: To Be Determined List

None at this time. All system requirements, external interfaces, and constraints have been successfully resolved for Version 1.0